

SVG 与 Canvas 路线分析报告

面向图模一体化能源系统建模平台

生成日期：2026-05-28

核心结论：该平台应采用“结构化图模数据为核心，SVG 负责可编辑图形表达，Canvas 只做性能型辅助层”的路线。

理由很直接：本平台不是普通展示大屏，而是专业图模一体化编辑器。它需要精确选择设备、端子、连接线、拐点、图层和参数表，还要支持右键菜单、拓扑检查、SVG/E 文件导出、人工布线和模型维护。SVG 的对象级表达更贴近这些需求；Canvas 更适合海量只读渲染和预览辅助。



1. 结论

对当前平台而言，SVG 主路线更加合适。Canvas 不建议替代主编辑画布，而应作为辅助渲染层，用于缩略图、背景层、大规模只读预览、导出预览或低频统计图层。

- 短期：继续保留 SVG 作为主编辑画布，重点优化增量计算、受影响连接线重布线、图层可见性和右侧面板渲染。
- 中期：引入 Canvas 作为辅助层，承担缩略图、背景、只读预览、热力图或大规模态势展示。
- 长期：支持“大模型只读 Canvas 模式 + SVG 精确编辑窗口”的混合架构。

判断依据是平台的核心操作对象不是像素，而是图元。设备、端子、母线、连接线、拐点、图层、参数字段和拓扑节点都需要长期保持可寻址、可编辑、可导出、可校验。

2. 为什么 SVG 更适合该平台

SVG 的优势不是单纯“能画图”，而是每个图形对象天然存在于 DOM 中。对图模一体化平台来说，这一点非常关键。

- 对象级编辑自然：设备、连接线、端子、母线、拐点都可以直接绑定事件和样式。
- 交互实现成本低：点击、双击、右键、拖拽、悬浮提示、选中态和禁用态都可以通过 DOM 与 CSS 直接表达。
- 与 SVG 文件目标一致：平台已经需要保存 SVG 显示参数，SVG 主画布天然适合导出和回显。
- 局部样式变化简单：开关开合、告警高亮、图层隐藏、端子重叠放大、能源颜色切换，都可以局部更新。
- 调试可视化友好：浏览器 DevTools 能直接检查某个图元的结构、属性和事件。

这些能力正好对应平台当前的主要需求：图上拓扑、联络线正交布线、端子吸附、图层管理、设备参数维护、图元树定位、E 文件导出和 SVG 显示保存。

3. Canvas 在该场景里的主要问题

Canvas 是像素画布。它的性能强，但对象不存在于浏览器 DOM 中。使用 Canvas 做主编辑器时，平台必须自行实现一整套对象系统。

- 需要自研命中检测：点击设备、端子、线段、拐点、母线区域和框选都要自行计算。
- 需要自研事件分发：不同图层、不同能源类型、选中/悬浮/禁用状态都要手动管理。
- 需要自研脏区渲染：否则每次拖拽都容易全量重绘，复杂场景下仍会卡顿。
- 需要额外实现 SVG 导出：Canvas 本身不能直接保留结构化矢量对象。
- 需要额外实现可访问调试能力：定位某个具体图元的问题比 SVG 更困难。

因此，如果直接把主编辑区改成 Canvas，短期可能获得一些渲染性能，但会显著增加连接线编辑、右键菜单、拓扑定位、参数表联动和图层操作的工程复杂度。

4. SVG 的短板与治理方向

SVG 的短板主要来自 DOM 数量、React 状态更新和全量计算，而不是 SVG 本身不能胜任。现阶段平台反复出现的卡顿，大多应优先从计算边界和渲染边界治理。

- 打开模型时，不应执行完整正交布线。已有路径可信时应直接显示。
- 图元移动后，只重布受影响连接线；不要让无关连接线参与路径搜索。
- 拓扑岛、拓扑节点、节点电压回填只在点击“图上拓扑”时计算。
- 图元树、参数表、拓扑告警应延迟渲染或虚拟化，避免大模型下拖慢主画布。
- 拖拽过程中使用 requestAnimationFrame 合并更新，拖拽结束后再做必要的轻量校验。
- 冷数据放到 ref 或外部 store，避免每次鼠标移动触发大范围 React 重渲染。

换句话说，平台需要遵守四句话：谁变了，只算谁；谁可见，只渲染谁；谁激活，只让谁响应事件；拓扑只在点击“图上拓扑”时计算。

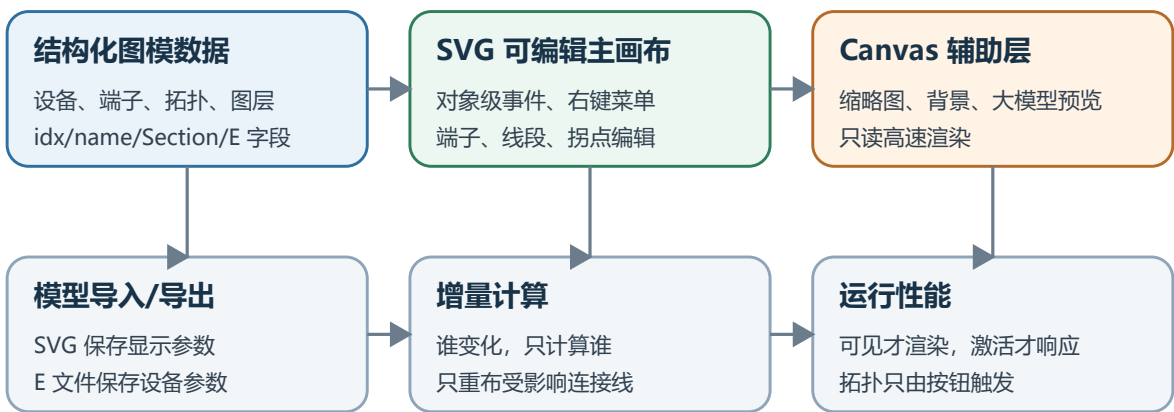
5. Canvas 的合理用途

Canvas 仍然有价值，但更适合做“性能型辅助层”，而不是替代 SVG 主编辑器。

- 大规模只读预览：例如几千到上万吨元的全景浏览、缩略图、鹰眼图。
- 背景层和热力图：例如电压等级色块、潮流热度、运行状态底图。
- 临时拖拽影子层：拖拽过程中将原位置虚影或高频动态效果放在 Canvas，可以降低 SVG DOM 更新压力。
- 导出预览层：在不影响可编辑 SVG 的前提下，快速生成截图或预览图。

Canvas 的优势应该用于高频、只读、像素级或者临时效果；涉及专业编辑、参数绑定和可导出结构时，仍应回到 SVG 或结构化数据。

6. 推荐架构



6.1 数据层：结构化模型是真源

平台的真源不应是 SVG DOM，也不应是 Canvas 像素，而应是结构化图模数据。设备、端子、连接线、图层、拓扑、Section 字段、idx/name 和导出规则都应在模型数据中维护。

6.2 编辑层：SVG 主画布

SVG 负责精确编辑和可视化表达，包括设备选择、端子拖拽、连接线正交路径、人工拐点、图层显示、右键菜单、图元树定位和参数联动。

6.3 Canvas 辅助层

Canvas 只处理不需要对象级 DOM 的场景，例如背景图、缩略图、热力图、临时拖拽影子和只读全景预览。它不承担模型真源，也不独立决定拓扑。

6.4 大模型模式

当模型规模很大时，可使用 Canvas 展示只读全局视图，用户选择局部区域后再切回 SVG 精确编辑。这样既保留专业编辑能力，也避免全量 SVG DOM 压力。

7. 推荐实施路线



短期重点

- 保留 SVG 主画布，不进行架构大切换。
- 打开模型时跳过完整正交布线，优先显示已保存路径。
- 移动、粘贴、删除、端子拖拽后，只处理受影响图元和连接线。
- 把保存、拓扑、导出和颜色配置等低频动作与拖拽高频动作解耦。

中期重点

- 增加 Canvas 缩略图或背景层，降低大范围预览成本。
- 建立空间索引，用于快速查询某图元附近的设备、母线和连接线。
- 图元树、告警列表、参数表做虚拟滚动和按需加载。
- 把正交路径搜索改为有边界的局部计算，避免全局扫描。

长期重点

- 提供大模型只读 Canvas 模式，作为浏览和定位入口。
- 局部精确编辑仍使用 SVG，保证端子、拐点、右键菜单和参数联动稳定。
- 将拓扑、布线、导出等重计算模块放入 Web Worker 或独立服务，避免阻塞 UI 线程。

8. 关键性能优化清单

- 连接线路径优化：从“全局最优”改为“受影响连接线局部最优”。
- 布线触发条件：只在端点、障碍物或图层可见性真正变化时触发。
- 状态管理：拖拽中的坐标使用轻量临时状态，落点后一次性提交模型状态。
- 渲染管理：不可见图层不渲染，不激活图层不响应事件。
- 面板管理：右侧参数表不要跟随鼠标移动实时重算。
- 拓扑管理：节点、拓扑岛、电压回填只由“图上拓扑”按钮触发。
- 撤销栈管理：切换模型后清空当前撤销栈，避免跨模型快照污染。
- 导出管理：导出 SVG/E 文件从模型数据生成，不反向依赖页面当前渲染状态。

9. SVG 与 Canvas 对比表

维度	SVG	Canvas
对象级编辑	强，天然 DOM 对象	弱，需要自研对象系统
点击设备/端子/线段	容易，事件可直接绑定	需要命中检测
右键菜单	容易，基于对象弹出	需要命中检测和事件分发
拖拽端子	容易，适合精确交互	需要自研拖拽和状态管理
连接线拐点编辑	容易，线段和控制点可对象化	需要自研几何编辑器
CSS 样式	强，适合状态和主题切换	弱，需手动绘制
图层显示隐藏	容易，可按组控制	需要重绘管理
导出 SVG	天然支持	需要另写导出器
大规模渲染性能	中等，需增量和虚拟化	强
平移缩放流畅性	中等，需优化 DOM 和事件	强
代码复杂度	较低，更贴合当前平台	较高，工程封装重
适合专业编辑器	很适合	需要大量工程封装
适合海量只读图	一般	很适合

10. 什么时候才考虑 Canvas 主路线

只有当平台主要目标从“编辑图模”转为“海量只读展示”，并且用户不再频繁编辑端子、拐点、参数、图层和拓扑关系时，才值得考虑 Canvas 作为主路线。

- 模型规模达到数万级图元，且大部分操作只是浏览、缩放、平移和查看状态。
- 右键菜单、端子调整、人工拐点、图元树联动等编辑需求明显减少。
- 系统有能力维护完整自研对象系统、命中检测、事件分发、导出器和调试工具。
- 导出 SVG 不再是核心产物，或者已经有独立矢量导出模块。

当前平台仍处于专业建模编辑器路线，因此这些条件并不成立。Canvas 可以引入，但不应替代 SVG 主画布。

11. 最终建议

推荐路线：继续以 SVG 作为主编辑画布，以结构化模型数据作为唯一真源，同时逐步增加 Canvas 辅助层。不要把性能问题简单归因于 SVG，也不要急于迁移到 Canvas 主路线。当前更重要的是消除全量计算、全量渲染和无边界路径搜索。

工程判断上，平台下一阶段应围绕“增量化、局部化、按需化”推进：打开时不重布线，拖拽时只更新必要图元，落点后只优化受影响连接线，拓扑只按用户命令计算，图层不可见则不参与渲染和操作。这样能在保留 SVG 编辑优势的同时，把大规模模型下的打开速度和操作速度提升到可接受范围。